Two-Host Investigation Runbook

Method, decision gates and the operating sequence for a live Windows + Linux host investigation

Commands are in the **Execution Reference**. Disk tool walkthroughs are in the **FTK Runbook** and **Autopsy Runbook**. Record everything in **Hunt Notes**. The theory behind collection, baselining and intel-driven hunting is in the **Analyst Methods Handbook**.

Scenario assumed: one Windows 10 workstation and one Linux LAMP server, live access, Volatility / FTK / Autopsy on the bench, findings due at the end.

Part 1 — The method

Methodology for live host analysis across a Windows workstation and a Linux server using Volatility, FTK, and Autopsy. The procedure is a fixed order of investigation; tools exist only to answer it.

The three rules

Acquire before alter. Memory is captured from both hosts and hashed before any disk is touched. Every executed command modifies the state being examined, and volatile memory holds the evidence most likely to carry an active intrusion — running code, open sessions, injected regions, decrypted material. Disk-first handling destroys that evidence before it is read.

Two independent sources or it is not confirmed. A single indicator is a lead. A finding requires two sources that agree and do not derive from each other: a process and its on-disk file, an event record and a matching artifact, a connection in memory and the same connection in a log. The term *confirmed* applies only when both are present. Everything else remains a lead until corroborated or discarded.

State the reasoning. Each action is prefaced with the expected result and followed by what the output changed in the working picture. The method is the deliverable; output volume is not.

The five questions

Host analysis answers five questions in fixed order. The order reflects evidence volatility: the first three are answered in memory, then proven on disk; the last two are answered on disk. The questions are platform-independent. Only the commands differ between Windows and Linux.

```
Q1  What is running that should not be?
Q2  What is talking that should not be?
Q3  How does it survive a reboot?
Q4  Who did it, and how did they get in?
Q5  When did it start?
```

The questions chain. A process identified in Q1 holds a connection in Q2, writes a file, registers persistence in Q3, executed under an account in Q4, and first appeared at a point in time in Q5. Each answer supplies the pivot for the next, and the chain reconstructs the intrusion end to end rather than producing isolated indicators.

Memory answers Q1 through Q3 with higher fidelity than a live host, because a process cannot conceal its own memory pages from the kernel structures Volatility parses, whereas a tool running on the live host can be subverted by code already resident on it. Disk answers Q4 and Q5 and corroborates Q1 through Q3. Analysis proceeds from most volatile to least volatile.

Phase 0 — Orient

Record before collecting: hostname, IP, OS build, and privilege level on each host. Confirm what was provided — live shell, memory capture, disk image — and the tool versions on the bench. Identify the authorized output location; evidence media is never written to. Set the clock reference to UTC and open a findings log before the first command.

State the plan: memory from both hosts, five questions on each, disk proof, no confirmation without a second source. This fixes the method before any result exists.

Phase 1 — Acquire

Windows memory: winpmem or FTK Imager memory capture, hashed on output. Linux memory: AVML or LiME, hashed on output. If no Volatility symbol profile exists for the running Linux kernel, build an ISF from the kernel banner with dwarf2json so the image is parsable. Disk follows: acquire and hash-verify with FTK Imager, or mount read-only. All analysis runs against copies; originals are preserved and the acquisition hash is the integrity reference.

Phase 2 — Work the five questions

Q1 — What is running that should not be

Enumerate processes by two methods and compare. `windows.pslist` walks the active process list; `windows.psscan` scans pool memory for process objects. A process present in `psscan` but absent from `pslist` was unlinked from the active list and is a primary lead. Examine process lineage with `windows.pstree` and `windows.cmdline`; parentage identifies anomalies faster than process names. High-signal lineage: Office or a browser spawning a command interpreter, a web service process spawning a shell, `svchost.exe` whose parent is not `services.exe`, `lsass.exe` whose parent is not `wininit.exe`. Run `windows.malfind` to locate private, committed, executable memory regions — the signature of injected or hollowed code. On the live host, flag binaries executing from user-writable paths and binaries with invalid or absent Authenticode signatures. An unexpected parent process is weighted above an unexpected process name.

Linux memory equivalents: `linux.pslist`, `linux.pstree`, `linux.malfind`, and `linux.check_modules` for hidden kernel modules. On the live host, resolve each suspect PID to its backing file through `/proc/<PID>/exe` and read `/proc/<PID>/cmdline`.

Q2 — What is talking that should not be

Pull network state from the memory image with `windows.netscan` and `linux.sockstat`, not only from the live host, because live-host network utilities report only what the host is willing to disclose. Target two conditions: a process holding a session inconsistent with its function, and traffic at a fixed interval. Beaconing presents as a repeating small exchange at a regular period; normal client traffic is irregular. Every external address is carried forward — a single actor across two hosts commonly reuses infrastructure, so an address seen on the workstation is checked against the server. The DNS client cache and the hosts file expose pinned or injected name resolution.

Q3 — How does it survive a reboot

Persistence establishes intent. Execution alone can be incidental; a persistence mechanism is deliberate. Windows locations, highest yield first: services with image paths outside `System32`, `SysWOW64`, and `Program Files`; scheduled tasks outside the `\Microsoft\` tree; the `Run` and `RunOnce` keys under HKLM and HKCU; WMI `__FilterToConsumerBinding` subscriptions. The common pattern is an innocuous administrative name whose action points to a script in a data directory. Corroborate against `windows.svcscan` in memory. Linux locations: cron directories and user crontabs, enabled systemd unit files, shell profile scripts, `/etc/ld.so.preload`, and `~/.ssh/authorized_keys`. A persistence entry whose target is the file identified in Q1 confirms the foothold.

Q4 — Who did it, and how did they get in

Shift from host to account. Windows logon events in the Security log distinguish local compromise from reach-in: type 2 interactive, type 3 network, type 10 remote interactive, event 4648 logon with explicit credentials, event 4672 assignment of administrative privileges. A network or explicit-credential logon from another host, or an account active outside its normal hours, indicates the workstation is not the origin and scope must extend outward. Enumerate local accounts and administrators group membership. Linux: `/var/log/auth.log` for accepted, failed, sudo, and session events; `last` and `lastb` for login records; any UID 0 account other than root. Where the entry vector was a web request rather than a credential, the auth records show no corresponding authentication, which is itself the finding.

Q5 — When did it start

The first-appearance time is anchored in execution artifacts before any file timestamp is trusted, because file timestamps are the first thing altered. Windows: Prefetch write times, Amcache first-execution records, Shimcache, UserAssist, and BAM/DAM, cross-checked against `$MFT` and the USN journal. When standard-information timestamps appear uniform or implausible, compare them to the filename-attribute timestamps in the MFT record; disagreement indicates timestomping, and the execution artifact becomes the authoritative time source. Linux: build a filesystem timeline with `fls` and

`mactime` and read it against the web and auth logs. The target time is the one preceding the alert that initiated the investigation; that interval is the portion of the intrusion not yet documented elsewhere.

Decision gates

A lead is promoted to a finding at a defined condition.

```
G1  Process unsigned, in a writable path, or with an invalid parent  -> acquire and hash the file
G2  External session with no business justification                  -> bind it to a process and a log
entry
G3  Persistence entry targeting that same file or script             -> foothold confirmed
G4  The executing account and its authentication method              -> extend scope to other hosts
G5  A first-seen timestamp preceding the alert                       -> intrusion start identified
```

Any gate that fires advances the chain to the next question. Breadth precedes depth: all five questions are cleared at a shallow level on both hosts before any single artifact is examined in depth, so the time budget is not consumed by one line of inquiry while the second host is unexamined.

Negative findings

Leads that fail to corroborate are documented, not discarded silently. A binary that appears suspicious but holds no connection, registers no persistence, and falls outside the established time window is recorded as unresolved after a bounded effort. An uncorroborated indicator is stated as uncorroborated. The distinction between a lead and a finding is the measured control.

Reporting

Each finding: plain text, UTC, hashed.

```
FINDING:   single line — what, where, first seen
SCOPE:     hosts - accounts - persistence and artifact locations - first/last seen UTC
EVIDENCE:  item - SHA-256 - source - collector and collection time
           corroborating source, independent of the first
SCOPE BASIS:  checks performed, and what evidence would change the conclusion
```

No finding is reported as confirmed on a single source. Scope — which hosts, which accounts — is established before remediation is discussed.

Time budget

Scaled to a 3–4 hour window, both hosts:

```
Orient and acquire (both hosts)       30 min    ~15%
Five questions, memory and live       80 min    ~40%
Disk proof and timeline               60 min    ~30%
Report and defend                     30 min    ~15%
```

Record the lead, continue, return. No single artifact justifies consuming the budget at the cost of coverage.

Part 2 — The operating sequence

The organizing idea. Under time pressure the failure mode is flailing between tools. A fixed sequence removes the "what next?" decision. You are only ever running the next step and stating why. Run the same order on both hosts:

```
   VOLATILE  ->  LIVE           ->  DEAD
   memory        running state      image + artifacts
   (acquire)     (triage)           (Autopsy/FTK, timeline)
         |            |                    |
         +---- record + hash as you go, corroborate ----+
                          |
                  finding record
```

Pre-flight — before the clock starts

Prove every tool works now, not when you are live on a box. A tool that fails under the clock is the most common avoidable loss.

```
[ ] Volatility 3: `vol -h` runs; `vol -f <img> windows.info` works on a test image
[ ] Vol2 fallback staged (version/profile mismatch is the usual reason you need it)
[ ] Linux memory symbols: dwarf2json present; plan to build the ISF from the
    kernel banner (`banners.Banners`) if no profile is on hand
[ ] FTK Imager launches (portable), can acquire + VERIFY
[ ] Full FTK: database up, case opens (if provided)
[ ] Autopsy opens; ingest modules present (Hash Lookup, Keyword, Recent Activity, Extension Mismatch)
[ ] Memory acquisition ready:  Windows -> winpmem or FTK Imager Capture Memory
                               Linux   -> AVML or LiME
[ ] Case folder + evidence naming set; sha256 ready
[ ] Hunt Notes open; both hosts added with their system time zones
[ ] Execution Reference open
```

State the method at the start:

> "Volatile first, then live triage, then disk. I keep the footprint small, record and hash as I go, and I

> confirm every finding with a second independent source before I call it."

Windows 10 sequence

```
MEMORY   acquire: winpmem | FTK Imager -> Capture Memory          (Exec Ref §3)
         vol3: windows.info, pslist/psscan/pstree, cmdline, netscan,
               malfind, svcscan, dlllist, handles, filescan, cmdscan/consoles   (§4)
         hunt: unlinked procs (pslist vs psscan), odd parents, injected regions,
               rogue services, C2 in netscan
LIVE     Q1 procs §1.2 -> Q2 net §1.3 -> Q3 persistence §1.4 -> Q4 accounts §1.5
         -> logs §1.7 -> Q5 execution artifacts §1.8
DISK     FTK Imager acquire + VERIFY | mount read-only
         FTK Runbook or Autopsy Runbook, Q1 -> Q5
         Q5 anchors: Prefetch, Amcache, UserAssist, BAM, $SI vs $FN
```

Linux (LAMP) sequence

```
MEMORY   acquire: AVML | LiME                                      (Exec Ref §3)
         vol3 + ISF: linux.pslist/pstree/psaux, bash, sockstat,
               malfind, check_modules/check_syscall, lsof         (§5)
LIVE     Q1 procs §2.2 (www-data children!) -> Q2 net §2.3 -> Q3 persistence §2.4
         -> Q4 accounts/auth §2.5 + web tier §2.7 + DB §2.8 -> Q5 file times §2.9
DISK     image + Autopsy; fls + mactime timeline (§2.9) read against
         access.log and auth.log; .bash_history, package logs, /etc changes
```

Each confirmed item = **event + on-disk artifact** (two independent sources), then a finding record.

Time discipline

```
[ ] Timebox each phase; check the clock at each host switch
[ ] Note a lead, move on, come back. Never rabbit-hole live
[ ] Record findings as you go. Do NOT leave reporting to the end
[ ] Breadth across both hosts before depth on one artifact
```

Tear-off

```
PRE-FLIGHT  Vol3(+Vol2), dwarf2json, FTK Imager, FTK db, Autopsy, mem-acquire; sha256; Hunt Notes
hosts+TZ
APPROACH    state method aloud; volatile -> live -> dead, same order on both hosts
QUESTIONS   Q1 running · Q2 talking · Q3 survives reboot · Q4 who/how in · Q5 when started
GATES       G1 hash it · G2 bind session to proc+log · G3 foothold · G4 extend scope · G5 start found
WINDOWS     mem(vol3) -> proc/net/persist/accounts/logs/exec-artifacts -> image + FTK/Autopsy ->
timeline
LINUX       mem(ISF) -> proc/net/cron/systemd/auth/web/db -> image + Autopsy + fls/mactime
REPORT      finding / scope / evidence + 2nd source / scope basis. UTC, SHA-256
```

> Defensive, authorized analysis on provided systems under rules of engagement. Acquire and hash before you alter.